MINI PROJECT REPORT

ON

SMART LOST & FOUND MATCHING SYSTEM

Using C Programming and Abstract Data Types

SUBMITTED BY	REGISTER NUMBER
N. Guruprasad	2025115035
A. Shahina Begam	2025115033
Sanjay K.	2025115003

DEPARTMENT	Information Science and Technology
INSTITUTION	Anna University – CEG
ACADEMIC YEAR	2026–2027

DEPARTMENT OF INFORMATION SCIENCE AND TECHNOLOGY

ANNA UNIVERSITY — COLLEGE OF ENGINEERING, GUINDY

ABSTRACT

The Smart Lost & Found Matching System is a menu-driven application developed in the C programming language. It helps maintain records of items reported as lost and items reported as found. The system compares item name, category, colour, location, and date to calculate a matching score and displays potential matches in descending order of score. It also supports item searching, claim submission, claim processing, and tracking of returned items. The project demonstrates the practical use of Abstract Data Types (ADTs), particularly linked lists, a priority queue, and a queue. The application uses dynamic memory allocation and standard C library functions and runs in a terminal.

1. INTRODUCTION

Lost personal belongings are common in colleges, libraries, laboratories, and other public places. When lost and found details are maintained manually, identifying the correct item can take time. This project provides a simple computerized method to register items and identify possible matches based on their details. It is designed as an educational mini project to demonstrate data structures and ADT operations using C.

2. PROBLEM STATEMENT

A manual lost-and-found register may make it difficult to compare many lost and found reports. Records may be overlooked, and claim handling may not follow a consistent order. A simple system is needed to store item details, rank possible matches, manage claims, and update an item's return status.

3. OBJECTIVES

To register lost items and found items with unique IDs.

To store item records using linked lists.

To compare item details and calculate a matching score.

To display potential matches from highest score to lowest score.

To search records by item name or category.

To manage claims using a first-in, first-out queue.

To mark items as returned and display returned-item records.

To demonstrate ADT concepts using standard C.

4. SCOPE OF THE PROJECT

The application is intended for a small campus or organization. It supports local registration, searching, matching, and claim processing during a single program session. The current version stores records in memory; therefore, the records are cleared when the program exits. It is a rule-based matching tool and does not verify the identity or ownership of a claimant.

5. EXISTING SYSTEM

Lost and found details may be written in a paper register.

Staff or students must manually compare item descriptions.

Potential matches can be missed when many records exist.

Claim processing and return-status updates may be difficult to track.

6. PROPOSED SYSTEM

Store lost and found records in linked lists.

Compare five fields: name, category, colour, location, and date.

Calculate a score out of 100 using predefined weights.

Use a priority queue to list higher-scoring matches first.

Use a queue to process submitted claims in FIFO order.

Update the found item status after a claim is processed.

7. SOFTWARE AND HARDWARE REQUIREMENTS

7.1 Software Requirements

Programming language: C

Compiler: GCC (MinGW on Windows, or GCC on Linux)

Operating system: Windows or Linux

Editor/IDE: VS Code, Code::Blocks, Dev-C++, or any text editor

7.2 Hardware Requirements

Computer or laptop

Basic keyboard and display

A C compiler installed on the system

8. ABSTRACT DATA TYPES USED

8.1 Linked List

A linked list stores item records as dynamically allocated nodes. Each item node contains an ID, name, category, colour, description, location, date, return status, and a pointer to the next node. Separate linked lists are maintained for lost items and found items.

8.2 Priority Queue

Potential matches are stored in a priority-ordered linked list. Each match contains pointers to a lost item and a found item, along with a score. A higher score receives higher priority, so the results are displayed in descending order.

8.3 Queue

Claims are stored in a FIFO queue. The first submitted claim is processed first. The queue uses front and rear pointers, allowing a claim to be added at the rear and removed from the front.

9. MATCHING SCORE

The program assigns points when a field exactly matches, ignoring differences between uppercase and lowercase letters. The maximum score is 100.

Item name match: 30 points

Category match: 15 points

Colour match: 15 points

Location match: 25 points

Date match: 15 points

Interpretation used by the program: 70–100 = strong potential match; 40–69 = possible match; 1–39 = weak match. A score indicates similarity only and does not prove that two records refer to the same physical item.

10. MODULE DESCRIPTION

Register Lost Item: Accepts details of a lost item, assigns an ID, and inserts it into the lost-item linked list.

Register Found Item: Accepts details of a found item, assigns an ID, and inserts it into the found-item linked list.

Find Matching Items: Compares every eligible lost item with found items, calculates scores, and orders matches by priority.

Search Items: Searches both lists by exact item name or category.

View Records: Displays all lost or found item records.

Submit Claim: Adds a claimant name and found-item ID to the claim queue.

Process Claims: Processes claims in FIFO order and marks eligible items as returned.

View Returned Items: Displays found items marked as returned.

View Pending Claims: Displays claims waiting to be processed.

11. ALGORITHM

Start the program.

Display the main menu.

Read the user's menu choice.

If the choice is to register a lost or found item, collect details and insert the record into the appropriate linked list.

If the choice is to find matches, compare each active lost item with each active found item and calculate the score.

Insert each positive-score match into the priority queue in descending score order.

If the choice is to search, compare the requested name or category with stored records.

If a claim is submitted, add it to the rear of the claims queue.

When claims are processed, remove them from the front and update the item's return status when valid.

Repeat until the user selects Exit.

Free dynamically allocated memory and stop.

12. PROGRAM FLOW

Start

Display menu

Read choice

Perform selected operation: register / match / search / view / claim

Display result

Return to menu unless Exit was selected

Free memory and End

13. SAMPLE TEST CASES

Test	Input / Action	Expected Result
1	Register a lost wallet	Lost item is stored with a generated ID.
2	Register a found wallet with matching details	Found item is stored with a generated ID.
3	Find matches	The matching wallet pair receives a high score and is displayed.
4	Search by category	Records with the entered category are displayed.
5	Submit a valid claim	Claim is added to the pending queue.
6	Process claims	Claim is processed and item status becomes Returned.
7	View returned items	Returned item appears in the returned-items list.
8	Search for a nonexistent item	A no-records message is displayed.

14. ADVANTAGES

Simple and menu-driven interface.

No hash table or external library is required.

Uses dynamic memory allocation and pointers.

Ranks potential matches automatically.

Demonstrates multiple ADTs in one application.

Can be extended for use in a campus environment.

15. LIMITATIONS

Records are stored only in memory and are not saved after exit.

Matching uses exact field comparisons rather than advanced text similarity.

A high score is only a suggestion and is not proof of ownership.

The current program does not authenticate claimants.

Dates and descriptions are entered manually and are not validated in detail.

16. FUTURE ENHANCEMENTS

Save and load records using file handling.

Use partial keyword matching for descriptions.

Add date-range matching and more flexible location matching.

Add claimant verification and approval by an administrator.

Generate summary reports for lost, found, claimed, and returned items.

Add a graphical interface in a future version.

17. CONCLUSION

The Smart Lost & Found Matching System provides a practical demonstration of Abstract Data Types using C. Linked lists store item records, a priority queue ranks possible matches, and a FIFO queue organizes claims. The system reduces the effort required to compare lost and found reports and gives students practical experience with structures, pointers, dynamic memory allocation, searching, and basic scoring algorithms.